SAVEETHA SCHOOL OF ENGINEERING
C PROGRAMMING ASSIGNMENT – Logic Analysis and Design Challenges

Field	Entry
Student Name	BAVADARANI S
Register Number	1912260562
Course Code & Name	CSA0209 C PROGRAMMING
Slot	B
Assignment / Task Title	Logic Analysis and Design Challenges (Q1–Q7, 50 marks)
GitHub Repository URL	https://github.com/bavadarini28 /CSA0209-1912260562-BAVADARANI.S.git
Submission Date	12 NOVEMBER 2026

Q6 – Loops That Must Terminate: Loan Repayment Schedule

Loop design

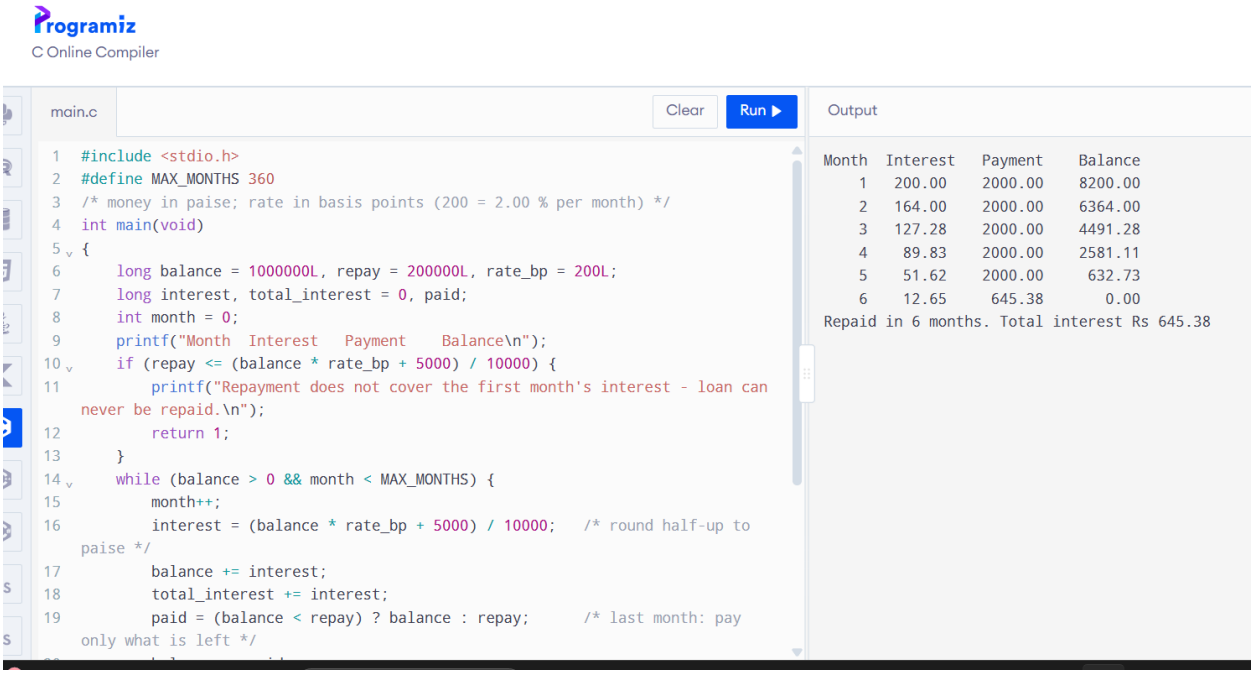
- **Which loop:** while. The number of months is not known beforehand (rules out for), and a loan with zero balance should print nothing (rules out do–while, which always executes once).
- **Termination:** while (balance > 0 && month < MAX_MONTHS). Money is in integer paise, so balance reaches exactly 0. With floating point, balance != 0 is dangerous: for example, 1.0 minus 0.1 ten times leaves 1.39 × 10^-16 in double (measured), never exactly 0, so the loop would not stop or would pay a 0.00000000000000014 instalment.
- **Last month:** pay min(balance, repayment). Otherwise the balance would turn negative.
- **Repayment ≤ monthly interest:** the balance never falls (it stays the same or grows), so the loop never ends. The program tests this before the loop and prints an error; the safety limit is a second line of defence.

```
#include <stdio.h>
#define MAX_MONTHS 360
/* money in paise; rate in basis points (200 = 2.00 % per month) */
int main(void)
{
   long balance = 1000000L, repay = 200000L, rate_bp = 200L;
   long interest, total_interest = 0, paid;
   int month = 0;
   printf("Month  Interest   Payment    Balance\n");
   if (repay <= (balance * rate_bp + 5000) / 10000) {
      printf("Repayment does not cover the first month's interest - loan can never be repaid.\n");
      return 1;
   }
   while (balance > 0 && month < MAX_MONTHS) {
      month++;
      interest = (balance * rate_bp + 5000) / 10000;   /* round half-up to paise */
      balance += interest;
      total_interest += interest;
      paid = (balance < repay) ? balance : repay;      /* last month: pay only what is left */
      balance -= paid;
      printf("%5d %5ld.%02ld %7ld.%02ld %7ld.%02ld\n", month, interest/100, interest%100, paid/100, paid%100, balance/100,
balance%100);
   }
```

```
    if (balance > 0) printf("Safety limit of %d months reached; balance Rs %ld.%02ld still due.\n", MAX_MONTHS, balance/100, balance%100);
    else printf("Repaid in %d months. Total interest Rs %ld.%02ld\n", month, total_interest/100, total_interest%100);
    return 0;
}
```

Output (Rs 10,000, 2% per month, Rs 2,000 repayment):

```
Month  Interest   Payment    Balance
    1  200.00    2000.00    8200.00
    2  164.00    2000.00    6364.00
    3  127.28    2000.00    4491.28
    4   89.83    2000.00    2581.11
    5   51.62    2000.00     632.73
    6   12.65     645.38       0.00
Repaid in 6 months. Total interest Rs 645.38
```



Correctness check by hand

Month 1: 10000 × 0.02 = 200; 10200 − 2000 = 8200. Month 2: 164; 8364 − 2000 = 6364. Month 3: 127.28; 6491.28 − 2000 = 4491.28. Month 4: 89.8256; 4581.1056 − 2000 = 2581.1056. Month 5: 51.6221; 2632.7277 − 2000 = 632.7277. Month 6: 12.6546; balance 645.3823 < 2000, so the last payment is Rs 645.38. Total interest = 645.38 (exact 645.3823) – matches the program (the program rounds each month's interest to the paisa, hence the tiny differences in the 4th digit). Check: 5 × 2000 + 645.38 = 10,645.38 = 10,000 + 645.38.

Growth investigation

Prediction: months $n = -\ln(1 - rP/E) / \ln(1+r)$ with rP = Rs 200 is the monthly interest on the first month; n grows without bound as $E \to 200$. Measured (P = 10,000, r = 2%):

Repayment E (Rs)	Months (loop)	Months (formula)
2000	6	5.32
1000	12	11.27
500	26	25.80
300	56	55.48
250	82	81.27

Repayment E (Rs)	Months (loop)	Months (formula)
220	122	121.09
205	188	187.53
201	268	267.81
200 or less	never ends	undefined

The count roughly doubles each time E − 200 halves; near the threshold a Rs 1 change in repayment changes the loan term by months.

Deployment decision

The limit is MAX_MONTHS = 360 (30 years: a longer micro-loan is not realistic). When it is reached the program prints: "Safety limit of 360 months reached; balance Rs … still due." and exits with an error status, not a "fully repaid" report. Defence: silent truncation would give a false schedule; the message tells the group leader to raise the repayment. Cases with repayment ≤ interest are rejected up front with a clear message, so the limit is only reached by very low (but valid) repayments.

Alternative Concept and Complexity Analysis

Closed form: months = ceil(−log(1 − r·P/E) / log(1 + r)) in O(1) time and O(1) space, versus O(n) iterations (6 here; up to 360). Precision: the formula uses floating-point logs, so the result is fractional and must be rounded up; it ignores the per-month rounding to paise, so the result can be off by a month at the boundary; it is undefined when r·P/E ≥ 1 (log of zero or a negative number) and loses accuracy when r·P/E is close to 1. It also cannot give the schedule, the final payment or the total interest without more work. The loop is exact in integer paise and produces the whole schedule, at a cost that is trivial for n ≤ 360.